PENSJONISTENE
digital agency

Process Guide

Contents

1 Introduction **2**

2 Code **3**
2.1 Running a Scrum meeting . . . 3
2.2 Working with the application . . . 3
2.3 From Scrum meeting to merge . . . 3
2.4 Branch Strategies . . . 4
2.5 Create a local branch . . . 4
2.5.1 Pulling updates from `main` . . . 4
2.5.2 Making new branches . . . 4
2.6 Local testing . . . 4
2.6.1 Running tests . . . 4
2.6.2 Running End-To-End (E2E) Tests . . . 5

3 Commit **6**
3.1 Adding and committing . . . 6
3.1.1 Rebasing to get updates from `main` . . . 6
3.2 Pushing to Gitlab . . . 6
3.2.1 Automatic triggering of merge requests . . . 6
3.2.2 Manually triggering a merge request in GitLab . . . 6
3.3 Merge conflicts . . . 9
3.4 Deleting branches . . . 10
3.5 Main as a protected branch . . . 10

4 Pre-Deploy **11**
4.1 Writing tests . . . 11
4.1.1 Pytest . . . 11
4.1.2 Playwright . . . 11
4.1.3 Viewing test results . . . 11
4.2 When a merge request fails . . . 11
4.3 Building images . . . 12
4.4 Making changes to the workflow . . . 12

5 Publish **13**
5.1 From Merge to live: Publishing to Staging and Production . . . 13
5.2 Manual Approval in Production Pipelines . . . 13
5.3 Accessing Namespaces via IP and Deploying from GitLab . . . 14

6 Monitoring Deployments **17**
6.1 Using `kubectl` to Monitor Kubernetes Resources . . . 17
6.1.1 Using `talosctl` to Monitor Node and System Health . . . 18
6.2 Practical Monitoring Flow . . . 19
6.2.1 Conclusion . . . 20

1 Introduction

The following guide explains the processes involved in a typical usage of the DevOps solution.

A typical usage on the solution is now given in summary:

1. The team holds a Scrum meeting to plan the work to be done over the next "iteration". Issues are assigned to team members.
2. A team member creates a local feature branch based of "main" to implement what is required from their assigned issue
3. If wished, the team member can run local automatic tests to ensure that changes do not break the overall solution
4. When the team member is confident about the new changes, he branch is pushed to the remote repository, and a merge request to main is started
5. The merge request runs tests and tries to merge back to main.
6. If the merge request is successful, the application is built and automatically deployed to the staging namespace
7. Team members can manually deploy the latest application image to production namespace

The following chapters expand on these points, highlighting key aspects of each element in this DevOps pipeline.

For questions, suggestions or comments, send an email to vionoreply@gmail.com.

2 Code

2.1 Running a Scrum meeting

Before any development work begins, a new sprint should be planned using GitLab's Scrum tools. Refer to the Setup Guide for details on configuring the project with a product backlog, issue boards, and sprint (iteration) planning.

Weekly planning meetings (held on Mondays) should follow the Scrum methodology. During the meeting, the team reviews the product backlog and selects relevant issues to include in the upcoming sprint. These issues are then added to the active iteration and assigned to developers based on priority and capacity.

The meeting should be held by a designated Scrum Master, who is responsible for managing the sprint cadence, monitoring team progress, and maintaining clarity in GitLab's project environment. Each meeting begins by reviewing the previous sprint and formally closing it before planning the next one.

Issues in GitLab can also include sub-tasks to help developers break down larger user stories into smaller, manageable work units. This improves tracking and ensures all parts of a task are completed.

- **Epics** represent long-term features or goals, typically spanning one or a few sprints
- **Issues** represent user stories or features that can be completed within a sprint, typically within a day
- **Tasks** within an issue represent concrete steps that can be completed in hours or a day.

2.2 Working with the application

For more information about setting up the application for local devlopment, see the setup guide.

2.3 From Scrum meeting to merge

Once the sprint has been planned and issues have been assigned during the Scrum meeting, the development process begins. This section visualizes the steps a developer follows to take an issue from the planning board to completed code merged into the main branch.

- Step 1: Create a local branch
- Step 2: Implement and test locally
- Step 3: Commit and push
- Step 4: Create a merge request
- Step 5: Code review and CI/CD
- Step 6: Automatic deployment

All of these steps are described in more detailed later in the Process Guide.

2.4 Branch Strategies

To align with DevOps practices, an Agile work method is suggested. One of the best practices, and that seems most suitable for the team size is the 'Trunk-Based Development'[1].

Trunk-Based Development is a branching strategy where all developers merge their work into a shared branch (typically `main`) at least once a day. Short-lived branches and frequent integrations reduce the likelihood of complex merge conflicts, and support fast feedback cycles. Following this strategy enables continuos integration (CI) and lays the foundation for continuos delivery (CD), who are both core practices in DevOps [1].

Challenges with Trunk-Based Development may include maintaining code stability during frequent integrations and ensuring sufficient automated testing. These can be addressed by implementing robust test suites and ensuring that code is always production-ready before merging[1].

2.5 Create a local branch

2.5.1 Pulling updates from `main`

To fetch and merge the latest changes from the remote `main` branch into your local `main` branch, use the following command:

```
git pull origin main
```

Make sure you are currently on the `main` branch before running this command.

2.5.2 Making new branches

To create a new branch and switch to it, use the following command from the branch you want to branch off from:

```
git checkout -b 'branch-name'
```

This will both create a new branch called `branch-name` and check it out (switch to it). Proceed to code the desired feature implementation.

2.6 Local testing

Test automation is key practice for reviewing and validation of the product.

2.6.1 Running tests

Before pushing your code, you should run local tests to ensure everything works as expected. By installing project dependencies using `pip install -r requirements.txt`, test packages are also installed.

From the root directory of the project, run the following command:

```
pytest -v
```

This command runs all the tests in the project. The `-v` flag enables detailed output to the terminal, listing each test by name and showing whether it passed or failed. This makes it easier to identify exactly which tests are running and how they perform.

To run tests in a specific folder, specify the path like this:

```
pytest -v tests/[NAME_OF_FOLDER]
```

Note that a test server should be running for E2E tests. See section 2.6.2 for details on running the provided test server.

2.6.2 Running End-To-End (E2E) Tests

End-to-End testing is done to verify complete user flows in an isolated environment without affecting the development database.

Run E2E tests from the root directory of the project, while the application is running in a test mode:

```
python run_test_server.py
```

3 Commit

3.1 Adding and committing

It is suggested to commit changes to code often. To do so, one may do the following (whilst in their own feature branch): Once the feature is implemented and tested, changes are committed and pushed to GitLab.

```
git add .
git commit -m "Appropiate commit message"
```

3.1.1 Rebasing to get updates from `main`

Changes can have been published to main whilst a team member is implementing a feature. To update their current branch with the latest changes from `main`, a team member can run the following commands:

```
git checkout main
```

```
git pull origin main
```

```
git checkout 'your-branch-name'
```

```
git rebase main
```

This should be done before pushing a branch to the remote repository or triggering a merge request from the command line, otherwise merge conflicts can arise.

3.2 Pushing to Gitlab

3.2.1 Automatic triggering of merge requests

When a developer has finished implementing desired changes or new features in their feature branch, they can push changes to the remote repository and trigger a merge request to main automatically with the following command:

```
    git push \
  -o merge_request.create \
  -o merge_request.target=main \
  -o merge_request.title="[Appropiate title]" \
  -o merge_request.description="[Appropiate description]" \
  -o merge_request.merge_when_pipeline_succeeds \
  -o merge_request.merge_train \
  origin my-feature-branch

```

Team members are encouraged to update their local main branch first and rebase their feature branch against main so that merge conflicts during a merge request can be avoided.

3.2.2 Manually triggering a merge request in GitLab

A team member can push their local branch to the GitLab repository using the following command:

`git push origin [NAME_OF_BRANCH]`

This branch will now be accessible in GitLab. To trigger a merge request, do the following:

1. Go to "Code" and press "Merge Requests"
2. Choose source branch and target branch, and continue with the merge request process
3. Add title and description of changes as needed, and trigger merge request
4. Press "set to auto-merge" to add merge request to merge train

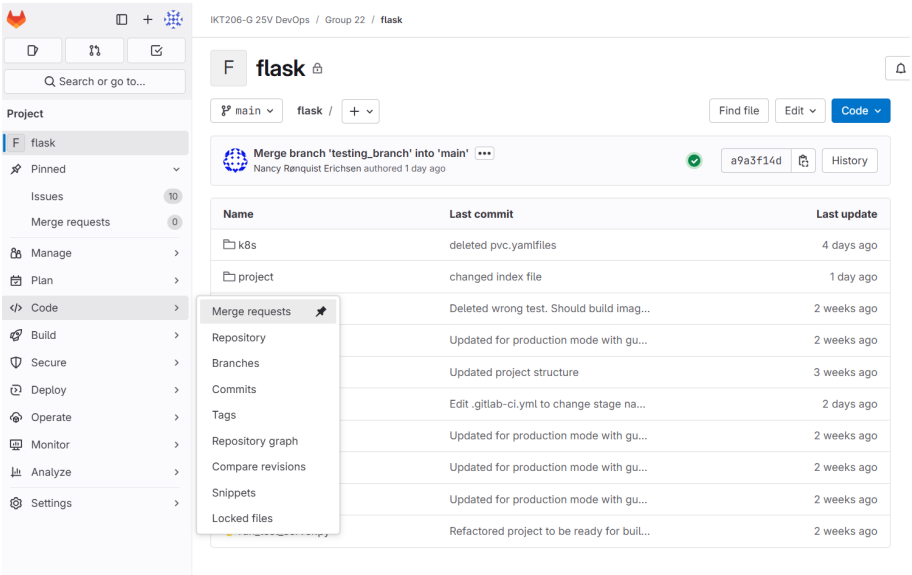

Figure 1: Merge requests can be found under "Code"

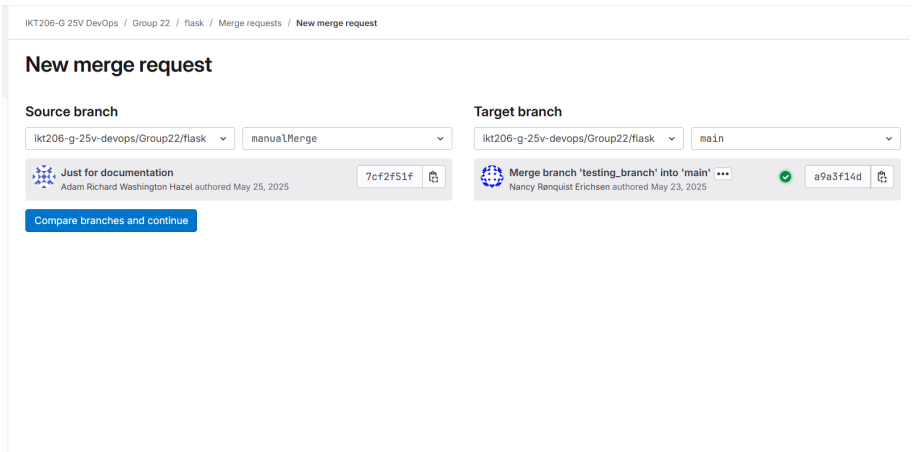

Figure 2: Choose source and target branch for merge request

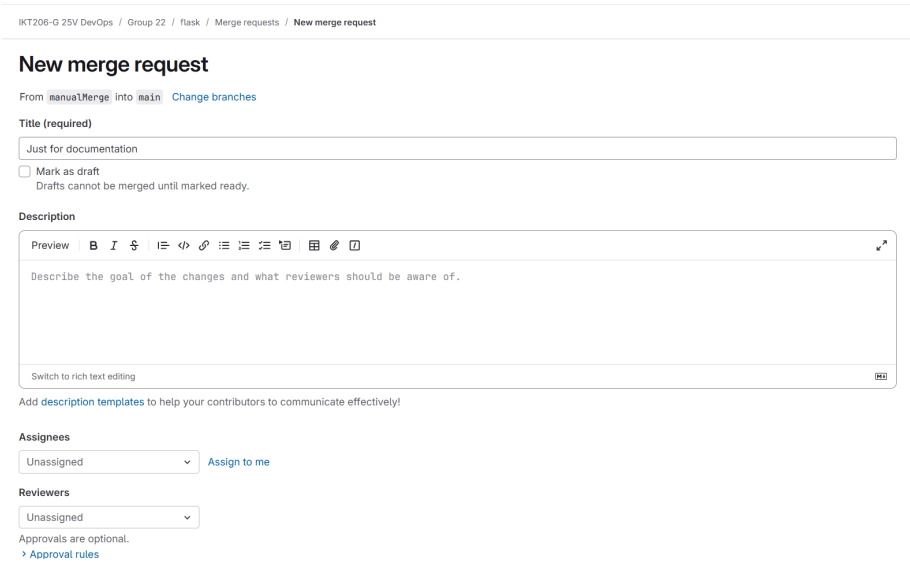

Figure 3: Fill in details for merge request so that other team members know what the merge request is for

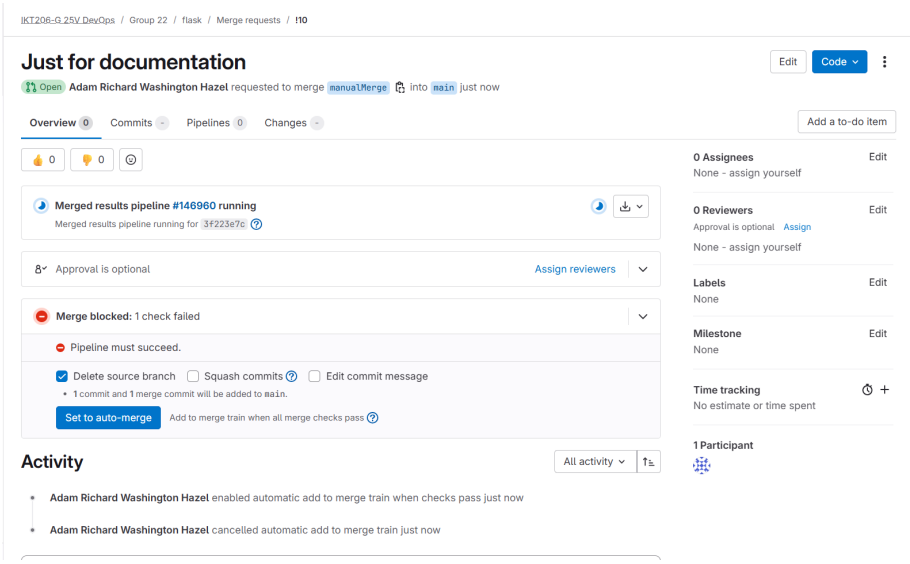

Figure 4: Set to auto-merge to add merge request to merge train

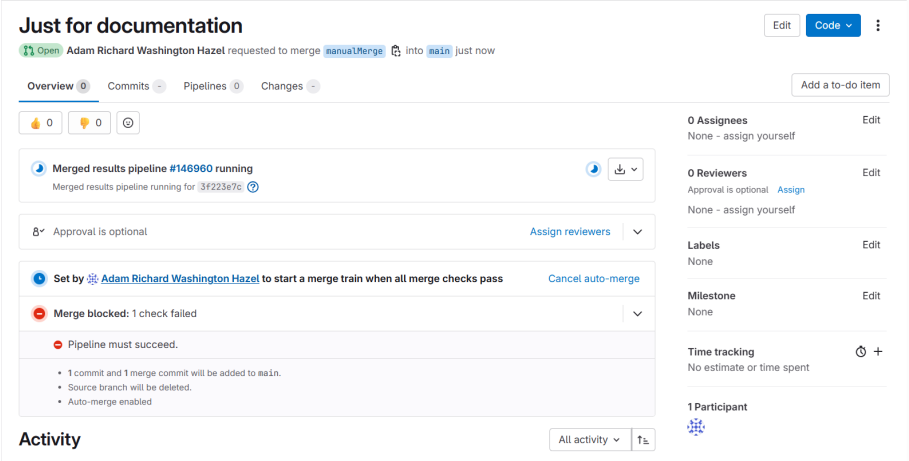

Figure 5: Merge request triggers GitLab workflow (including automatic tests)

3.3 Merge conflicts

Merge conflicts occur when Git cannot automatically merge changes from two different branches because the same lines in the same file have been modified in both branches. This is a common situation when multiple developers work on the same parts of the codebase simultaneously.

To minimize the likelihood of merge conflicts, the team should:

- Pull and rebase frequently from the `main` branch
- Keep feature branches short-lived
- Communicate clearly when working on shared files

If a merge conflict does occur during a merge or rebase, Git will pause the operation and mark the conflicting sections in the files. These conflicts must be resolved manually by editing the affected files, removing the conflict markers, and choosing which changes to keep.

Example of a conflict marker:

```
<<<<<<< HEAD
current code in main
=======
incoming code from your branch
>>>>>>> my-feature-branch
```

After resolving the conflicts:

```
git add <resolved-files>
git rebase --continue    # if you're rebasing
# or
git commit               # if you're merging manually
```

If needed, it is also possible to abort a rebase or merge attempt:

```
git rebase --abort
# or
git merge --abort
```

By integrating often and resolving small conflicts early, the team can avoid large, time-consuming conflicts later in the process.

3.4 Deleting branches

To delete a branch locally, run the following command:

```
git branch -d 'branch-name'
```

This will delete the branch only if it has already been merged. To force delete a branch (even if it hasn't been merged), use:

```
git branch -D 'branch-name'
```

To delete a branch on the remote repository, run:

```
git push origin --delete 'branch-name'
```

This will remove the branch from the remote (GitLab).

3.5 Main as a protected branch

Main is a protected branch in GitLab, meaning that only team members with "Maintainer" or "Owner" status can push directly to main. For changes by other team members to be included in main, the team member must use a merge request.

4 Pre-Deploy

When a merge request is triggered, the GitLab workflow (found in `./gitlab-ci.yml`) is triggered.

The workflow has three stages:

1. test
2. build
3. publish

The build and publish stages are not run until a merge is successfully completed.

Although team members do not need to take any active actions until the "publish" stage (where a team member can decide to publish the latest image to the production namespace), the team may wish to expand the test coverage by writing more tests, view test results, or handle a failed merge request.

To view the progress of the triggered workflow, click "Build" → "Pipelines".

The source branch in the merge request is automatically deleted when a merge request is successful.

4.1 Writing tests

4.1.1 Pytest

See pytest documentation [2] for more information regarding writing unit and integration tests.

4.1.2 Playwright

E2E tests are written using the Playwright framework. See Playwright documentation for writing E2E tests [3].

4.1.3 Viewing test results

See the setup guide for more details on how to view test results.

4.2 When a merge request fails

If a merge request fails, it is most commonly due to one or more of the following:

- Failing unit, integration, or end-to-end tests
- Syntax errors or linting issues
- Merge conflicts with the `main` branch
- Deployment script errors in the CI pipeline

GitLab automatically stops the pipeline and marks the merge request as failed. The developer is notified directly in the GitLab interface and can review detailed logs for each job in the pipeline.

To fix the issue:

1. Review the failed job logs in the "CI/CD > Pipelines" section of GitLab

2. Identify the cause of the failure (e.g. a broken test or missing dependency)
3. Make the necessary changes locally
4. Push the updated branch again to re-trigger the pipeline

If the failure was caused by a merge conflict or outdated branch, the developer should rebase their branch against `main` and resolve any conflicts before pushing:

```
git checkout main
git pull origin main
git checkout feature/my-branch
git rebase main
# resolve conflicts if needed
git push
```

Once the issue is resolved and the pipeline passes, the merge request will proceed automatically, or can be merged manually if required.

4.3 Building images

The "build" stage of the workflow runs a job that builds the application into an image that is saved in the GitLab container image registry.

Built images can be found at the following path:

Deploy → Container Registry

4.4 Making changes to the workflow

The workflow can be edited, but the team is encouraged to engage in a carefully planned process when changing the workflow to ensure that the needed jobs are still performed.

5 Publish

5.1 From Merge to live: Publishing to Staging and Production

Once a merge request is accepted into the `main` branch, the CI part of the pipeline is automatically triggered. This pipeline consists of two main phases:

1. **Build phase** – The Flask application is packaged into a container image and pushed to the internal container registry.
2. **Publish phase** – The generated image tag is injected into the relevant `flask-deployment.yaml` manifest files located in the `k8s/staging` and `k8s/production` directories.

The deployment to `staging` occurs automatically as part of the pipeline. This ensures that the latest version of the application on `main` is always available in a live test environment.

The deployment to `production`, on the other hand, is intentionally gated behind a manual approval step. This additional control helps prevent accidental releases and ensures that changes intended for production have been explicitly reviewed.

In GitLab, under the `Build > Pipelines` section, the team can view an overview of recent pipelines. By clicking on a specific pipeline, a visual representation of the build and publish stages is displayed, as shown in Figure 6.

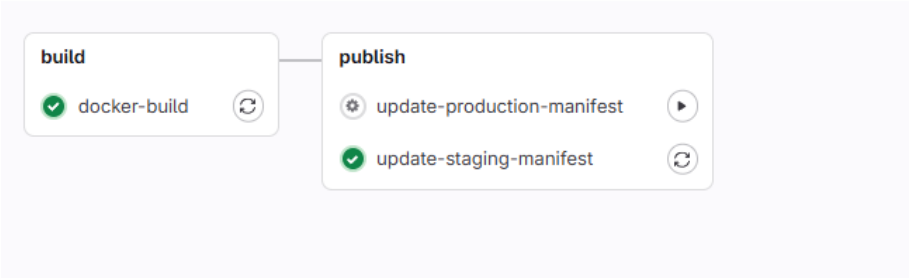

Figure 6: Overview of the CI/CD pipeline stages: automatic deployment to `staging` and manual approval for `production`.

5.2 Manual Approval in Production Pipelines

To keep things safe and intentional, the last step in the CI/CD pipeline—`update-production-manifest`—is set to require manual approval. This makes sure that production doesn't get updated automatically without someone giving it a final check.

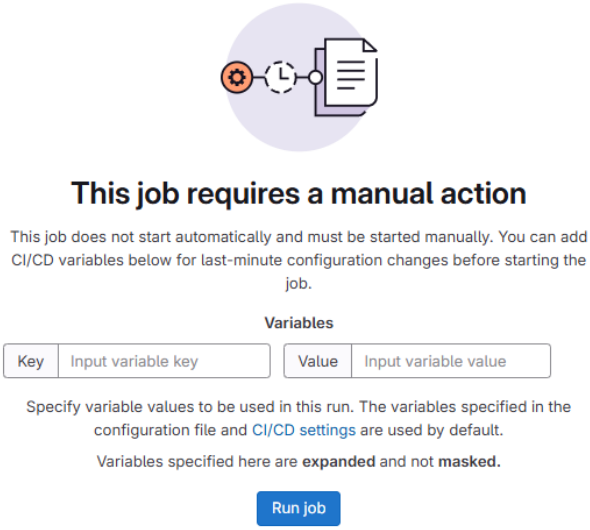

Figure 7: Manual confirmation required before pushing to production

After verifying that everything looks good in staging, a team member can go into GitLab, open the latest pipeline, and hit the `Run job` button for the production update. This works as a final checkpoint—it gives the team time to double-check that everything is working as expected before moving to production. Once the job is confirmed and runs successfully, the production image gets deployed too.

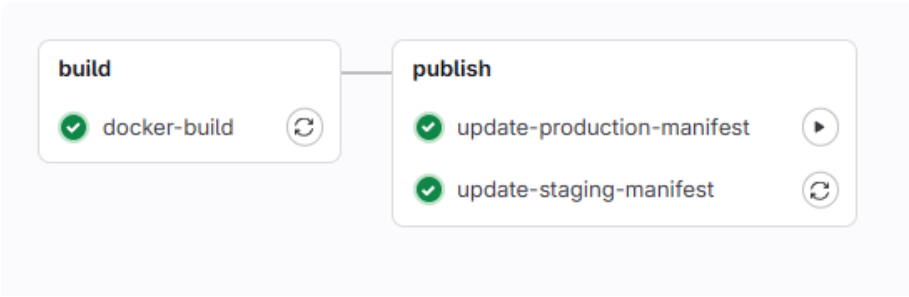

Figure 8: Both staging and production deployments completed

This way, the team keeps full control over what goes live, while still benefiting from automation for everything leading up to it.

5.3 Accessing Namespaces via IP and Deploying from GitLab

Both the `staging` and `production` namespaces have their own running version of the Flask application. These are exposed externally using Kubernetes `NodePort` services, making it easy to test

and inspect deployments straight from the browser.

Each environment is mapped to a specific port on every node in the cluster, as defined in the respective `flask-service.yaml` files:

- `http://<node-ip>:30080` for `staging`
- `http://<node-ip>:30081` for `production`

The port number is specified under the `nodePort` field in the service configuration file. This is described in more detail in the setup guide under the Kubernetes section. To verify which port is currently exposed in the cluster, the following command can be used:

```
kubectl get service flask-api-service -n staging
kubectl get service flask-api-service -n production
```

To find the IP address of a node, use:

```
kubectl get nodes -o wide
```

This shows a list of nodes and their internal IPs, which can be plugged directly into a browser address bar along with the right port. There's no need for an ingress controller, the NodePort service handles external access in this setup.

The `staging` version will always show the latest image built from `main`, while `production` might lag behind if manual approval hasn't been triggered yet. This makes it easy to visually confirm the state of each environment and double-check that everything behaves as expected. As shown in Figures 9 and 10, the new image is already live in the `staging` environment, but it has not yet been promoted to `production`. If a team member now triggers a manual deployment as described in Section 5.2, the production instance will be updated to match the state of staging.

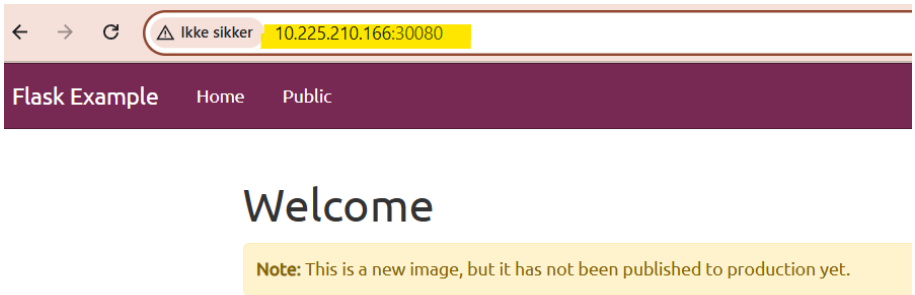

Figure 9: Flask application in `staging` — latest image from CI

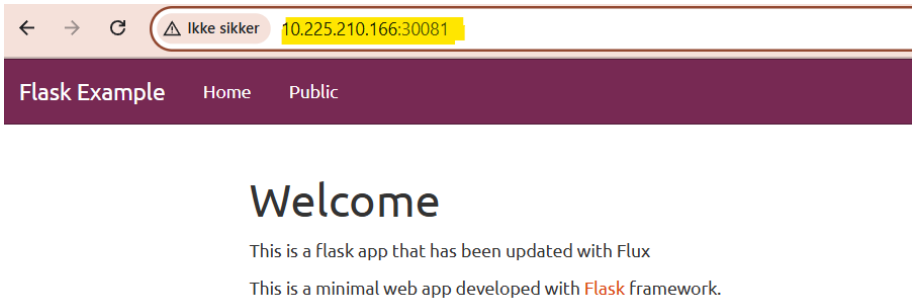

Figure 10: Flask application in `production` — manually approved deployment

6 Monitoring Deployments

After changes are pushed to GitLab and deployed to either the `staging` or `production` environments, it is important that the team monitors the state of the system to ensure all components are functioning correctly. Kubernetes and Talos provide powerful command-line tools for this purpose: `kubectl` and `talosctl`. This section outlines how these tools can be used effectively by team members to observe system health, inspect logs, and troubleshoot potential issues. Instructions for installing and configuring these tools are provided in the setup guide.

6.1 Using `kubectl` to Monitor Kubernetes Resources

`kubectl` provides insight into the state of workloads running in Kubernetes. Many of the following commands are also referenced in the setup guide, where they are used by the team to validate the successful deployment and functionality of the Flask application.

Check pod status

To verify that all relevant pods are running, use the following command:

```
kubectl get pods -n <namespace>
```

This command lists all pods in the specified namespace, showing their readiness state, status, number of restarts, and age.

Monitor rollout progress

To track the progress of a deployment rollout and confirm that new pods are created and ready, use the following command:

```
kubectl rollout status deployment/<deployment-name> -n <namespace>
```

This command waits until the rollout is complete and prints a confirmation message. It is useful after applying a new deployment or updating an existing one.

Inspect logs from a deployment

To view logs from a running application (such as the Flask API), use the following command:

```
kubectl logs deployment/flask-api -n <namespace>
```

To follow the logs in real time, add the `-f` (follow) flag:

```
kubectl logs deployment/flask-api -n <namespace> -f
```

This is useful for verifying that the application starts correctly and is serving traffic.

Describe a failing pod

If a pod fails to start, crashes repeatedly, or remains in a pending state, use the following command to get detailed status and event information:

```
kubectl describe pod <pod-name> -n <namespace>
```

This command displays the full lifecycle state of the pod, including container startup attempts, mounted volumes, environment variables, and recent events such as failed probes or missing secrets.

Check service accessibility

To verify that services are properly exposed within the cluster and (optionally) to the outside world, use the following command:

```
kubectl get services -n <namespace>
```

This will list all services in the specified namespace, including their type, internal IP, exposed ports, and age.

```
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
flask-api-service   NodePort    10.[redacted]   <none>        80:30081/TCP   2d4h
postgres            ClusterIP   None            <none>        5432/TCP       29h
```

Figure 11: Example output from `kubectl get services` showing a NodePort and ClusterIP service

In the example above, the `flask-api-service` is exposed via a `NodePort` (port `30081`), making it accessible from outside the cluster using the IP address of any Kubernetes node. The PostgreSQL service, on the other hand, uses a `ClusterIP` with no external access, which is typical for internal database services.

Use the external node IP and port to test connectivity from a browser or terminal:

```
curl http://<node-ip>:<nodePort>
```

6.1.1 Using `talosctl` to Monitor Node and System Health

While `kubectl` is focused on Kubernetes workloads, `talosctl` provides low-level access to the Talos operating system and node-level services.

Check overall cluster health

Use the `health` command to check the readiness and health of all Talos-managed nodes and Kubernetes control plane components:

```
talosctl --nodes <node-ip> health
```

This command performs a comprehensive health check across all system components, including etcd, kubelet, apid, and Kubernetes static pods. It waits for all nodes and control plane services to report as healthy and schedulable. If everything is good, the boot and health check sequence returns `OK`, including disk and memory validation, etcd clustering, control plane pod status, and node schedulability. This confirms that the Talos nodes are fully operational and that the Kubernetes control plane is healthy.

Inspect system service logs

To view logs for specific Talos services (e.g., containerd, kubelet, etcd), use:

```
talosctl --nodes <node-ip> logs <service-name> --tail 50
```

Common service names include:

- `containerd` – manages containers and images
- `kubelet` – runs and monitors pods on the node
- `etcd` – stores Kubernetes state (on control plane nodes)
- `apiserver` – serves the Kubernetes API (on control plane nodes)

To stream logs continuously:

```
talosctl --nodes <node-ip> logs kubelet --follow
```

6.2 Practical Monitoring Flow

After pushing a change or applying a deployment, a typical monitoring workflow might look like:

1. Use `kubectl get pods -n staging` to ensure new pods are running
2. Monitor rollout with `kubectl rollout status deployment/flask-api -n <namespace>`
3. Stream logs using `kubectl logs -f deployment/flask-api -n <namespace>`
4. Use `talosctl health` to verify that the cluster nodes are healthy
5. Check container runtime logs with `talosctl logs containerd -tail 50`
6. Test the service externally via browser or `curl`

Using K9s to Monitor the Cluster

While `kubectl` and `talosctl` provide powerful CLI-based monitoring, there are also several third-party tools that can enhance visibility and usability such as `k9s`. K9s is a terminal-based user interface for interacting with Kubernetes clusters[4]. K9s is a terminal-based tool that provides a fast and interactive way to monitor and manage Kubernetes resources.

The following points describe how team members can use K9s effectively as part of the deployment and monitoring workflow[5]:

Start K9s

Launch the interface by running `k9s` in a terminal. By default, it connects to the current kubeconfig context.

Navigate between namespaces

Press `:ns` and select either `staging` or `production` to focus on the relevant environment. This isolates the view to resources in that namespace.

Monitor pod status

From the default view (pods), monitor the state of the application by checking the STATUS, RESTARTS, and AGE columns. Healthy pods should show `Running` with zero restarts.

View logs

Select a pod and press `l` to stream logs from the container. This is useful for verifying behavior after deployment or identifying errors.

Inspect resources

Use shortcut keys (e.g., `d` for Deployments, `s` for Services) to switch views. Press `Enter` to drill into individual resources for more details.

Filter and search

Use `/` to filter the current view by name or label. This is helpful in larger namespaces with many resources.

Restart or delete pods

Select a pod and press `r` to restart, or `Ctrl+D` to delete. This can help resolve transient issues, but should be done with caution in production.

K9s continuously watches the cluster and updates the view in real time, making it a powerful tool for day-to-day monitoring and troubleshooting.

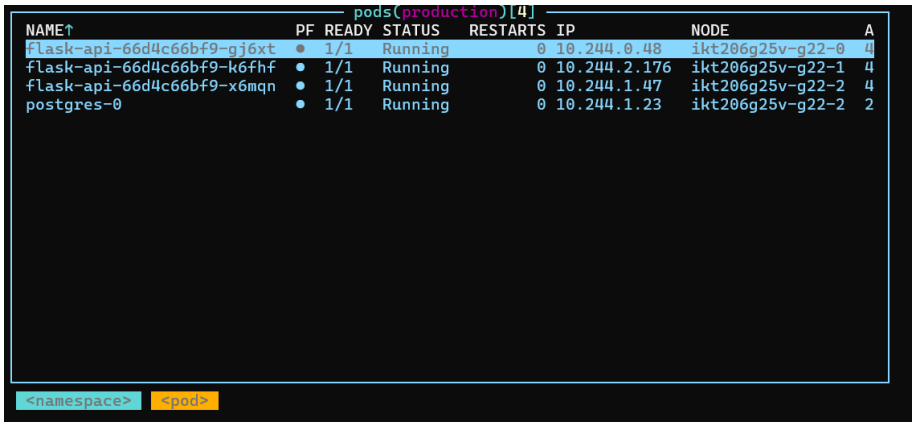

Figure 12: Using k9s, monitoring the pods in production namespace

Monitoring is important after deploying to production. Even if deployments are successful in staging, unforeseen issues such as missing secrets, configuration mismatches, or resource constraints can occur[6].

6.2.1 Conclusion

By combining `kubectl` for Kubernetes-level insights and `talosctl` for node-level diagnostics, the team has full visibility into the state of the system.

In addition, tools like `k9s` can be valuable for live monitoring and interaction with cluster resources in real time. While this guide highlights core tools, there are many other useful solutions available for teams looking to expand their operational toolkit.

References

[1] *Trunk-Based development*, [Online; accessed 22. May. 2025]. [Online]. Available: https://dora.dev/capabilities/trunk-based-development/.

[2] *pytest documentation*, [Online; accessed 19. May 2025], Mar. 2025. [Online]. Available: https://docs.pytest.org/en/stable.

[3] *Installation | Playwright*, [Online; accessed 19. May 2025], May 2025. [Online]. Available: https://playwright.dev/docs/intro.

[4] *K9s - Manage Your Kubernetes Clusters In Style*, [Online; accessed 23. May 2025], Apr. 2025. [Online]. Available: https://k9scli.io.

[5] *Commands*, [Online; accessed 23. May 2025], Apr. 2025. [Online]. Available: https://k9scli.io/topics/commands.

[6] *What is Kubernetes monitoring?* [Online; accessed 22. May 2025], May 2025. [Online]. Available: https://www.elastic.co/what-is/kubernetes-monitoring.